

Final Project – Deep Learning

Matching CV / Offres d'emploi

par Deep Siamese Bi-LSTM

Conducted at Université Paris-Dauphine PSL

by

Evelina Spac – Nine Reinhart

15 Mai, 2026

CONTENTS

1	Introduction	2
2	Construction du Dataset	2
2.1	Prétraitement du texte (fichier <code>dataset.ipynb</code>)	2
2.2	Labellisation	3
2.3	Limites	3
3	Baselines	4
3.1	TF-IDF + Régression Logistique	4
3.2	Siamese CNN	4
3.3	CNN + Attention Layer	5
4	Modèle Principal : Siamese Bi-LSTM	5
4.1	Principe du réseau siamois	5
4.2	LSTM Bidirectionnel	5
4.3	Préparation des données	6
4.4	Architecture	6
4.5	Fonction de perte : Contrastive Loss	7
4.6	Décision finale	7
4.7	Entraînement	7
4.8	Extension : Siamese LSTM avec Attention	8
5	Expériences et Résultats	8
5.1	Tableau comparatif	9
5.2	Analyse des résultats	9
5.3	Limites	9
5.4	Perspectives	10
6	Conclusion	10
7	Annexes	11
7.1	Siamese Bi-LSTM Implémentation - Code	11
7.2	Attention Layer	11
7.3	TF-IDF	11
7.4	Similarité cosinus	12
7.5	F1-score macro	13

1 Introduction

Le recrutement est un processus coûteux et chronophage : un recruteur passe en moyenne plusieurs heures à lire des CV pour chaque poste ouvert, avec un risque important de biais ou d'erreurs d'appréciation. L'automatisation du matching entre un CV et une offre d'emploi constitue donc un enjeu pratique majeur, à l'intersection du traitement automatique du langage (NLP) et de l'apprentissage automatique.

La difficulté principale provient de la nature hétérogène des deux documents : un candidat peut être parfaitement qualifié pour un poste sans pour autant utiliser les mêmes mots-clés ou formulations que ceux présents dans l'offre d'emploi.

Les approches traditionnelles basées sur des représentations de type Bag-of-Words ou TF-IDF reposent principalement sur la fréquence des mots et les correspondances lexicales directes. Bien qu'efficaces pour certaines tâches de NLP, elles peinent à capturer les relations sémantiques profondes entre deux textes exprimant une même idée avec des vocabulaires différents. Ces méthodes restent donc limitées pour comprendre le sens global d'un CV ou d'une offre d'emploi.

Afin de dépasser ces limitations, nous proposons une approche fondée sur un réseau siamois à Bi-LSTM, capable d'apprendre une représentation sémantique des CV et des offres d'emploi dans un espace latent commun. Grâce au partage de poids entre les deux branches du réseau et à l'utilisation d'une fonction de perte contrastive, le modèle apprend à rapprocher les paires compatibles et à éloigner les paires incompatibles dans l'espace de représentation. Contrairement aux méthodes purement lexicales, cette approche permet de mieux prendre en compte le contexte, l'ordre des mots et les similarités sémantiques entre les documents.

Nous construisons notre propre dataset en combinant des CV issus d'un dataset HuggingFace et des offres d'emploi provenant d'un dataset Kaggle. La labellisation des paires est réalisée automatiquement à partir d'un seuil de similarité cosinus TF-IDF, conformément à plusieurs travaux de la littérature. Nous comparons ensuite différentes approches, allant de modèles classiques de machine learning jusqu'à notre architecture siamoise Bi-LSTM, inspirée des travaux de Sachin Maheshwary and Vinod Misra (*Matching resumes to jobs via deep siamese network* [1]).

2 Construction du Dataset

Notre dataset est construit à partir de deux sources complémentaires.

CV : Nous utilisons le dataset *datasetmaster/resumes* disponible sur HuggingFace, qui après suppression des duplicats contient 847 CV de profils variés dans le domaine tech (Software Engineer, Data Analyst, Cybersecurity Engineer, Frontend Developer, etc.). Ce dataset combine des CV réels et synthétiques normalisés dans un schéma JSON commun, conçu pour des tâches NLP de matching.

Offres d'emploi : Nous utilisons un dataset Kaggle (*training_data.csv*) contenant initialement 853 offres d'emploi. Il s'agit d'un jeu de données déjà structuré, comprenant pour chaque offre un titre de poste, un nom d'entreprise, et une description textuelle. Nous avons ensuite filtré les offres pour ne conserver que les postes du domaine tech, aboutissant à 229 offres tech retenues sur 853. Ce filtre est justifié par l'alignement avec les CV disponibles, qui sont tous des profils tech.

2.1 Prétraitement du texte (fichier `dataset.ipynb`)

Avant toute modélisation, chaque texte (CV et offre d'emploi) est nettoyé et normalisé selon le pipeline suivant, implémenté dans la fonction `preprocess_text` :

1. **Expansion des contractions anglaises** (you're → you are, don't → do not, etc.) via regex.
2. **Conversion en minuscules.**
3. **Suppression des colonnes inutiles et des caractères non-alphabétiques** (chiffres, ponctuation) via regex.

4. **Suppression des stop words** anglais issus de `sklearn` (`ENGLISH_STOP_WORDS`), en conservant `not`, `no`, `nor` pour ne pas perdre les négations.

Pour les CV, les champs structurés (expériences, projets, éducation, compétences, certifications, préférence de localisation) sont extraits via des fonctions dédiées (`extraire_experience`, `extraire_projets`, `extraire_education`, `extraire_skills`, `extraire_certifications`) et concaténés dans un champ unifié `cv_text`.

Pour les offres, le champ `job_text` est construit par concaténation du titre du poste, du nom de l'entreprise et de la description textuelle nettoyée.

2.2 Labellisation

En l'absence d'annotations humaines, nous générons les labels de façon automatique par seuillage de la similarité TF-IDF cosinus, approche validée dans la littérature ou encore utilisée par ConFit [2] pour ses tâches de classification de CV/Offres.

Le pipeline de labellisation est le suivant :

1. **Vectorisation TF-IDF** de tous les textes (CV + offres) sur un vocabulaire commun (*voir Annexe*).
2. **Calcul de la matrice de similarité cosinus** ($847 \times 229 = 193\,963$ paires évaluées) (*voir Annexe*).
3. **Attribution des labels par seuillage** : $\text{score} \geq 0.045 \rightarrow \text{label } 1$; $\text{score} \leq 0.017 \rightarrow \text{label } 0$; zone intermédiaire : paire rejetée.
4. **Équilibrage du dataset** : sous-échantillonnage aléatoire des paires négatives pour avoir un dataset composé de 1491 paires avec `label=1`, et 1491 paires avec `label=0`.

Dans l'étape 2, les labels sont générés automatiquement à partir de seuils empiriques sur les scores de similarité :

- si $\text{score} > 0.07$: la paire est considérée compatible ($\text{label} = 1$) ;
- si $\text{score} < 0.013$: la paire est considérée incompatible ($\text{label} = 0$) ;
- sinon : la paire est jugée ambiguë et supprimée du dataset.

Ces seuils ont été choisis après analyse de la distribution des similarités :

- le seuil positif correspond approximativement aux meilleurs scores observés ;
- le seuil négatif correspond à des documents très peu similaires.

Cette stratégie permet de construire automatiquement un dataset supervisé sans annotation manuelle, tout en limitant le bruit dû aux paires ambiguës.

2.3 Limites

On pourrait objecter que TF-IDF, utilisé ici pour générer les labels, est précisément le modèle que [1] critiquent pour son incapacité à capturer les *underlying semantics*. Cette apparente contradiction est en réalité pleinement justifiée.

TF-IDF est utilisé ici comme outil de construction du dataset, non comme modèle de matching. Un score cosinus TF-IDF élevé signifie que le CV et l'offre partagent des termes importants et rares, condition nécessaire mais non suffisante de compatibilité. Il constitue une borne inférieure conservative : les labels sont bruités mais défendables comme proxy de la compatibilité lexicale.

Si le Siamese Bi-LSTM surpasse la baseline TF-IDF sur l'ensemble de test même entraîné sur des labels générés par TF-IDF, cela prouve qu'il a appris quelque chose de plus profond que la simple correspondance de mots.

3 Baselines

Nous comparons plusieurs modèles de Machine Learning et Deep Learning. Tous les modèles sont entraînés sur 70 % des données (soit 2087 paires), validés sur 10 % (soit 298 paires) et évalués sur 20 % (soit 597 paires). L'ensemble de validation est utilisé pour la sélection du meilleur modèle (early stopping) ainsi que pour le choix du seuil de décision, tandis que l'ensemble de test est exclusivement réservé à l'évaluation finale rapportée dans le Tableau 4.

3.1 TF-IDF + Régression Logistique

Chaque paire (CV, offre) est représentée par la différence absolue de leurs vecteurs TF-IDF : $\mathbf{X} = |\mathbf{h}_{cv} - \mathbf{x}_{job}|$. Cette représentation capture les écarts terme à terme entre les deux documents. Une régression logistique est entraînée sur ce vecteur différence, en utilisant les mêmes train set et test set que pour les modèles Deep Learning, garantissant une comparaison équitable.

3.2 Siamese CNN

Notre deuxième modèle baseline est un réseau siamois utilisant un CNN comme encodeur. Il s'inspire directement du papier de référence, qui propose une adaptation siamoise des réseaux convolutionnels pour le matching CV/offre d'emploi. Comme pour le Bi-LSTM, les deux branches du réseau encodent séparément le CV et l'offre, puis une distance euclidienne est calculée entre les deux vecteurs obtenus.

3.2.1 Architecture

Notre implémentation repose sur une classe `CNN` composée d'une couche d'embedding suivie de plusieurs blocs convolutionnels. Les deux entrées (CV et offre d'emploi) passent par exactement le même encodeur, grâce à la fonction `encode()` utilisée dans les deux cas.

Avant d'entrer dans le réseau, les textes sont convertis en tenseurs numériques selon le pipeline suivant : construction du vocabulaire (Dictionary), tokenisation de chaque texte, et padding à `MAX_LEN = 200` tokens. Chaque texte est ainsi transformé en séquence d'indices, puis converti en vecteurs denses via la couche d'embedding. Ces vecteurs sont ensuite transmis aux blocs convolutionnels.

Dans la méthode `forward()`, les représentations du CV ($\mathbf{h}_{cv}$) et de l'offre ($\mathbf{h}_{job}$) sont calculées indépendamment mais avec les mêmes poids :

- `h_cv = self.encode(cv)`
- `h_job = self.encode(job)`

Table 1: Architecture du Siamese CNN

Couche	Description	Dimension de sortie
Entrée	Séquence d'indices (paddée à 200)	<code>batch_size</code> × 200
Embedding	<code>nn.Embedding(V , 200)</code> from scratch	<code>batch_size</code> × 200 × 200
Conv1D + LeakyReLU + MaxPool (×3)	64 filtres, noyau 10, pool 3	<code>batch_size</code> × 64 × L'
Flatten + Linéaire	Projection sémantique	<code>batch_size</code> × 256
Distance	$d = \ \mathbf{h}_{cv} - \mathbf{h}_{job}\ _2$	<code>batch_size</code> × 1

3.2.2 Entraînement

- Fonction de perte : Contrastive loss
- Optimiseur : Adam ($\text{lr} = 10^{-4}$)
- Batch size : 32
- Epochs : 30 avec sauvegarde du meilleur modèle sur la validation loss

- Embedding : `nn.Embedding(|V|, 200)`, apprise from scratch

3.3 CNN + Attention Layer

Pour aller plus loin que la baseline du papier de référence [1], qui n'utilise pas de mécanisme d'attention, nous avons également implémenté une version augmentée du CNN avec une couche d'attention.

Dans le CNN classique, la représentation finale du document est obtenue en aplatissant toutes les sorties des blocs convolutionnels puis en les projetant via une couche dense. Ce procédé traite toutes les positions de la séquence de manière uniforme. Dans notre architecture, la couche d'attention est appliquée après les blocs convolutionnels, sur les sorties temporelles du réseau. Les scores d'attention sont normalisés à l'aide d'une fonction softmax afin d'obtenir une distribution de poids sur la séquence. La représentation finale du document est ensuite obtenue comme une somme pondérée des sorties convolutionnelles, à laquelle on applique une moyenne pour produire un vecteur unique par document.

L'architecture reste donc identique à celle du CNN classique pour les blocs convolutionnels (3 blocs Conv1D + LeakyReLU + MaxPool, embedding de dimension 200, noyau 10, 64 filtres), la seule différence étant le remplacement de l'étape *flatten + linear* par la couche d'attention suivie d'une projection finale dans l'espace sémantique de dimension 256.

4 Modèle Principal : Siamese Bi-LSTM

Notre modèle principal s'inspire de [1], qui utilise un réseau siamois, mais au lieu d'utiliser un CNN, nous avons décidé d'utiliser un LSTM bidirectionnel.

4.1 Principe du réseau siamois

Un réseau siamois est une architecture de réseau neuronal composée de deux (ou plusieurs) sous-réseaux identiques partageant les mêmes poids et paramètres. Il est conçu pour comparer deux entrées et déterminer leur similarité.

Dans notre cas, les deux branches du réseau encodent séparément le CV et l'offre d'emploi sous forme de vecteurs sémantiques. La distance entre ces deux représentations est ensuite calculée afin d'estimer la compatibilité entre le candidat et le poste.

Le partage des poids est fondamental : il garantit que les CV et les offres sont projetés dans le même espace sémantique, ce qui rend leur comparaison cohérente, tout en réduisant le nombre total de paramètres à apprendre de moitié.

4.2 LSTM Bidirectionnel

Notre adaptation remplace le CNN du papier de référence [1] par un LSTM bidirectionnel.

Dans le cours, nous avons étudié les LSTM classiques, qui constituent une amélioration des réseaux de neurones récurrents (RNN) traditionnels. Contrairement aux RNN simples, les LSTM sont capables de conserver des dépendances à long terme grâce à un mécanisme de mémoire interne contrôlé par différentes portes (input gate, forget gate et output gate). Cette architecture permet de limiter le problème du vanishing gradient et d'exploiter plus efficacement le contexte d'une séquence textuelle.

Dans le cadre du traitement de texte, un LSTM lit les mots séquentiellement afin de construire une représentation du document dépendant du contexte précédent, comme générer le prochain mot par exemple. Cependant, un LSTM classique ne parcourt la séquence que dans un seul sens, généralement de gauche à droite. Or, dans une phrase ou un document, le sens d'un mot peut dépendre à la fois des mots qui le précèdent et de ceux qui le suivent.

Pour cette raison, nous utilisons un Bi-LSTM, qui combine deux LSTM : l'un parcourant le texte de gauche à droite, et l'autre de droite à gauche. Les représentations produites par les deux réseaux sont ensuite concaténées afin d'obtenir une représentation plus riche et plus contextuelle du document. Cette

architecture est particulièrement adaptée à notre problème de matching CV/job, car les compétences, expériences ou technologies importantes peuvent apparaître à différents endroits du texte et leur interprétation dépend fortement du contexte global.

LSTM classique vs Bi-LSTM :

Table 2: Comparaison LSTM classique vs Bi-LSTM

	LSTM classique (cours)	Bi-LSTM (notre projet)
Direction	→ gauche à droite	→ et ← simultanément
Contexte	Seulement ce qui précède	Avant + après chaque mot
Représentation	n_{hid}	$n_{hid} \times 2$
Usage typique	Génération, traduction	Classification, résumé de doc

Le Bi-LSTM lit la séquence dans les deux sens en parallèle. Les derniers états cachés avant et arrière sont concaténés : $\mathbf{h} = [\mathbf{h}_{fwd}; \mathbf{h}_{bwd}] \in \mathbb{R}^{2 \cdot n_{hid}}$. En PyTorch : `bidirectional=True`.

4.3 Préparation des données

Avant d'entrer dans le réseau, les textes sont convertis en tenseurs numériques selon le pipeline suivant (identique aux TPs LSTM du cours) :

1. **Construction du vocabulaire (Dictionary)** : On parcourt l'ensemble du corpus (CV + offres) et on associe un indice entier unique à chaque mot, avec trois tokens spéciaux : `<empty>`, `<start>`, `<end>`.
2. **Tokenisation** : Chaque texte est converti en une séquence d'indices entiers (`LongTensor`).
3. **Padding** : Les séquences sont complétées avec `<empty>` jusqu'à `MAX_LEN = 200` tokens. Les séquences plus longues sont tronquées. On utilise `pack_padded_sequence` pour que le LSTM ignore le padding.

4.4 Architecture

Concrètement, notre implémentation repose sur une classe `SiameseLSTM` composée d'une couche d'embedding (`nn.Embedding`) suivie d'un Bi-LSTM (`nn.LSTM(..., bidirectional=True)`).

La couche `Embedding` est implémentée comme une `nn.Embedding` apprise par backpropagation. Matrice de poids de taille (`vocab_size × ninp`), initialisée aléatoirement et optimisée conjointement au reste du réseau.

Les deux entrées: le CV et l'offre d'emploi, passent par exactement le même encodeur, grâce à la fonction `encode()` utilisée dans les deux cas.

Chaque texte est d'abord transformé en séquence d'indices, puis converti en vecteurs denses via la couche d'embedding. Ces vecteurs sont ensuite transmis au Bi-LSTM, qui traite la séquence dans les deux directions (gauche → droite et droite → gauche). Le réseau retourne alors les états cachés finaux des deux directions :

- `h_fwd` : dernier état caché du LSTM avant
- `h_bwd` : dernier état caché du LSTM arrière.

Ces deux représentations sont concaténées avec : `torch.cat([h_fwd, h_bwd], dim=1)` afin d'obtenir un vecteur final de taille $2 \times n_{hid}$, contenant des informations contextuelles provenant des deux sens de lecture. Dans la méthode `forward()`, les représentations du CV (`h_cv`) et de l'offre (`h_job`) sont calculées indépendamment mais avec les mêmes poids :

- `h_cv = self.encode(cv, cv_len)`
- `h_job = self.encode(job, job_len)`

Le modèle mesure ensuite leur similarité à l'aide d'une distance euclidienne :

$$dist = torch.norm(h_{cv} - h_{job}, dim = 1)$$

Ainsi, plus la distance entre les deux vecteurs est faible, plus le modèle considère que le CV correspond à l'offre d'emploi. À l'inverse, une grande distance indique une faible compatibilité entre les deux documents.

Table 3: Architecture du Siamese Bi-LSTM

Couche	Description	Dimension de sortie
Entrée	Séquence d'indices (paddée à 200)	$batch_size \times 200$
Embedding	<code>nn.Embedding(V , 100)</code> from scratch	$batch_size \times 200 \times 100$
Bi-LSTM	<code>hidden_size=128</code> , 2 couches	$batch_size \times 256$
Représentation	$[h_{fwd}; h_{bwd}]$	$batch_size \times 256$
Distance	$d = \ h_{cv} - h_{job}\ _2$	$batch_size \times 1$
Décision	label = 1 si $d < \text{seuil}$	$batch_size \times 1$

4.5 Fonction de perte : Contrastive Loss

Contrairement à une classification classique avec BCE Loss, nous utilisons la **contrastive loss**, conçue spécifiquement pour les réseaux siamois :

$$\mathcal{L}(y, d) = y \cdot d^2 + (1 - y) \cdot \max(\text{margin} - d, 0)^2 \quad (1)$$

où $d = \|h_{cv} - h_{job}\|_2$ est la distance L2, $y \in \{0, 1\}$ le label, et $\text{margin} = 1.0$ un hyperparamètre.

- Si $y = 1$ (paire compatible) : la loss pénalise les grandes distances, forçant le réseau à *rapprocher* les représentations.
- Si $y = 0$ (paire incompatible) : la loss pénalise les distances inférieures à la marge, forçant le réseau à *éloigner* les représentations d'au moins **margin**.

Le paramètre **margin** définit la distance minimale que le modèle doit maintenir entre les représentations des paires incompatibles. Ainsi, si deux documents négatifs sont déjà séparés d'une distance supérieure à $\text{margin} = 1.0$, la loss devient nulle car le modèle les considère comme suffisamment différents.

4.6 Décision finale

Le modèle retourne une distance brute. La décision se fait par seuillage : $\hat{y} = \mathbf{1}[d < \text{seuil}]$. Le seuil optimal est déterminé sur le test loader en maximisant l'accuracy. Ici il correspond à un seuil de 0.6. Donc si $d < 0.6$, alors on attribue un *label* = 1 entre le CV et l'offre.

4.7 Entraînement

- Optimiseur : Adam ($\text{lr} = 10^{-3}$)
- Batch size : 32
- Epochs : 30 avec visualisation des courbes train/val loss
- Embedding : `nn.Embedding(|V|, 100)`, apprise from scratch

4.8 Extension : Siamese LSTM avec Attention

Nous avons également testé une extension de notre modèle principal en ajoutant une couche d'attention après le Bi-LSTM. L'objectif est de ne pas représenter le document uniquement à partir des derniers états cachés, mais de permettre au modèle de pondérer différemment les informations contenues dans toute la séquence.

Ainsi, les termes importants comme des compétences techniques (« *Python* », « *SQL* », « *Machine Learning* ») ou des expériences spécifiques peuvent avoir une contribution plus forte dans la représentation finale du CV ou de l'offre d'emploi.

Dans notre implémentation, la sortie complète du Bi-LSTM est d'abord récupérée avec `pad_packed_sequence`. Elle est de dimension `batch_size × seq_len × 2nhid`, car le LSTM est bidirectionnel. Cette séquence de représentations est ensuite transmise à une couche d'attention.

Notre couche d'attention repose sur trois projections linéaires : W_Q , W_K et W_V , correspondant respectivement aux requêtes, clés et valeurs. À partir de la sortie du Bi-LSTM, nous calculons donc :

$$Q = W_Q X, \quad K = W_K X, \quad V = W_V X$$

Les scores d'attention sont ensuite obtenus par produit matriciel entre Q et K :

$$scores = \frac{QK^T}{\sqrt{d_k}}$$

où $d_k = 256$ est la dimension interne utilisée dans la couche d'attention. La division par $\sqrt{d_k}$ permet de stabiliser les valeurs des scores avant l'application du softmax.

On applique ensuite un softmax afin d'obtenir des poids d'attention entre les différents tokens de la séquence:

$$A = softmax(scores)$$

Puis la représentation pondérée est calculée par :

$$out = AV$$

Enfin, nous effectuons une moyenne sur la dimension temporelle avec `out.mean(dim=1)` afin d'obtenir un seul vecteur final par document, de dimension `batch_size × 256`.

Dans le modèle `SiameseLSTMAtt`, cette couche d'attention est appliquée dans la fonction `encode()` après le Bi-LSTM. Le CV et l'offre d'emploi passent toujours par le même encodeur partagé : embedding, Bi-LSTM, puis attention. On obtient ainsi deux vecteurs, h_{cv} et h_{job} , dont la distance euclidienne est calculée comme dans le modèle principal :

$$d = \|h_{cv} - h_{job}\|_2$$

Cette extension permet au modèle de mieux exploiter l'ensemble des sorties du Bi-LSTM et de donner plus d'importance aux tokens pertinents du CV ou de l'offre, par exemple les compétences, les expériences ou les technologies mentionnées.

5 Expériences et Résultats

Tous les modèles sont évalués sur le même ensemble de test (20% des données). Les métriques rapportées sont : Accuracy, F1-score (macro), Precision, Recall. La accuracy est notre métrique principale.

5.1 Tableau comparatif

Table 4: Résultats comparatifs de tous les modèles sur l'ensemble de test

Modèle	Acc.	F1	Prec.	Rec.	AUC
TF-IDF + Régression Logistique	0.8322	0.8321	0.85	0.81	0.9177
Siamese CNN	0.8223	0.8223	0.78	0.91	0.9246
Siamese CNN + Attention	0.8540	0.8538	0.83	0.89	0.9352
Siamese Bi-LSTM	0.8473	0.8471	0.83	0.88	0.9336
Siamese Bi-LSTM + Attention	0.8506	0.8501	0.82	0.91	0.9403

5.2 Analyse des résultats

Les résultats du Tableau permettent plusieurs observations intéressantes sur la comparaison entre approches lexicales et approches sémantiques.

La régression logistique entraînée sur la différence absolue des vecteurs TF-IDF atteint une accuracy de 83.22%, ce qui constitue une baseline particulièrement solide. Ce résultat élevé n'est cependant pas surprenant : nos labels ayant été générés par seuillage de la similarité cosinus TF-IDF, un modèle exploitant ces mêmes représentations dispose d'un avantage structurel. Cette baseline représente donc la borne supérieure de ce qu'il est possible d'atteindre en se limitant aux correspondances lexicales directes.

Le Siamese Bi-LSTM (84.73%) surpasse légèrement le Siamese CNN (82.23%), confirmant l'intérêt d'une lecture séquentielle et bidirectionnelle du texte pour notre tâche. Le CNN, en appliquant des filtres convolutionnels locaux, capture efficacement les n-grammes mais reste sensible à la position des mots dans la séquence. Le Bi-LSTM, en revanche, peut accumuler des dépendances contextuelles à longue distance, ce qui est particulièrement pertinent pour des documents structurés comme un CV, où une compétence mentionnée au début (par exemple dans la section **skills**) peut être liée à une expérience décrite bien plus loin.

L'ajout d'une couche d'attention améliore systématiquement les deux architectures : +3.2 points pour le CNN (de 82.23% à 85.40%) et +0.3 point pour le Bi-LSTM (de 84.73% à 85.06%). L'apport plus important pour le CNN s'explique par le fait que celui-ci ne dispose d'aucun mécanisme intrinsèque de pondération temporelle, contrairement au Bi-LSTM qui exploite déjà son état caché final pour résumer la séquence. La couche d'attention permet ainsi au CNN de dépasser la simple agrégation uniforme par flatten et de se concentrer sur les tokens les plus discriminants (compétences techniques, intitulés de poste, technologies).

Au-delà de l'accuracy à seuil fixe, l'AUC fournit une mesure de la qualité du classement des paires indépendamment du seuil de décision. On observe que le Siamese Bi-LSTM + Attention atteint le meilleur AUC (0.9403), suivi du Siamese CNN + Attention (0.9352). Cela suggère que notre modèle principal apprend une représentation où les distances entre paires compatibles et incompatibles sont mieux séparées globalement, même si l'accuracy à seuil = 0.6 est légèrement plus faible que celle du CNN + Attention.

Conclusion intermédiaire. L'écart entre la baseline TF-IDF (83.22%) et notre meilleur modèle (85.06%) reste modeste en absolu, ce qui est attendu compte tenu du biais introduit par la labellisation automatique. Le point essentiel est que nos modèles siamois surpassent malgré tout la baseline TF-IDF sur l'ensemble de test, alors même qu'ils sont entraînés sur des labels générés par TF-IDF. Cela confirme que les représentations apprises par le Bi-LSTM et l'attention capturent des relations sémantiques qui dépassent la simple correspondance lexicale.

5.3 Limites

Notre approche de labellisation automatique par TF-IDF cosinus constitue une approximation des jugements humains. Comme le soulignent Sachin Maheshwary and Vinod Misra [1], TF-IDF souffre d'une

term specificity inhérente : il est aveugle aux synonymes et aux reformulations. Un CV mentionnant "développeur" et une offre mentionnant "ingénieur logiciel" peuvent être parfaitement compatibles mais obtenir un score TF-IDF proche de zéro.

Cette limite implique que notre dataset contient nécessairement des faux négatifs: des paires compatibles non détectées par TF-IDF. En pratique, si notre modèle surpasse malgré tout la baseline TF-IDF, cela prouve qu'il a capturé des représentations sémantiques plus riches.

5.4 Perspectives

Plusieurs pistes d'amélioration sont envisageables :

1. Annotation humaine d'un sous-ensemble, comme dans le papier de Sachin Maheshwary and Vinod Misra, pour valider et corriger les labels automatiques.
2. Utilisation d'une vraie base de données de CV, sans l'utilisation de Faker comme dans le cas du dataset trouvé sur HuggingFace.
3. Intégration de plusieurs features structurées (années d'expérience, niveau d'études) en complément du texte brut.
4. Utilisation de Sentence-BERT [3], un modèle pré-entraîné de représentation sémantique de phrases basé sur l'architecture Transformer, comme encodeur, qui atteint des performances state-of-the-art sur ce type de tâche.
5. Évaluation avec différents splits: Notre découpage actuel en train/val/test est réalisé aléatoirement sur les paires, ce qui signifie qu'un CV ou une offre présent dans le test peut également apparaître dans le train (sous forme de paires différentes). Cette configuration favorise artificiellement les performances du modèle, qui peut mémoriser certaines représentations vues à l'entraînement. Pour évaluer plus rigoureusement la capacité de généralisation du modèle, nous avons implémenté trois stratégies de splits inspirées de la littérature sur les systèmes de recommandation :
 - CV-cold : les CV du test set ne sont jamais vus à l'entraînement, mais les offres peuvent l'avoir été. Cela simule le scénario d'un nouveau candidat à matcher avec les offres existantes.
 - Job-cold : les offres du test set ne sont jamais vues à l'entraînement, mais les CV peuvent l'avoir été. Cela simule l'arrivée d'une nouvelle offre d'emploi à matcher avec la base de candidats existante.
 - CV/Job-cold : ni les CV ni les offres du test ne sont vus à l'entraînement. Cette configuration est la plus difficile et représente le cas le plus réaliste, mais elle réduit significativement la taille du dataset utilisable (1638 paires au lieu de 2982 dans notre cas).

6 Conclusion

Ce projet aborde le matching CV/offre d'emploi comme une tâche de classification binaire, en construisant un dataset original à partir d'un dataset HuggingFace (CV) et d'un dataset Kaggle (offres), avec labellisation automatique par seuillage TF-IDF cosinus, et en comparant des modèles de complexité croissante : des baselines ML jusqu'à quatre variantes de réseaux siamois (CNN, CNN+Attention, Bi-LSTM, Bi-LSTM+Attention).

La construction du dataset s'est avérée la partie la plus complexe du projet, impliquant de nombreuses décisions de conception (choix des sources, filtrage des offres non-tech, gestion des descriptions aberrantes, définition des seuils de labellisation) qui ont toutes été justifiées et documentées dans le fichier `dataset.ipynb`.

L'architecture Siamese Bi-LSTM représente une adaptation motivée du papier de référence : *Matching resumes to jobs via deep siamese network* [1], qui adresse directement une limitation soulevée par les auteurs eux-mêmes concernant les LSTM unidirectionnels. Le mécanisme d'attention constitue une extension naturelle vers les architectures Transformer.

7 Annexes

7.1 Siamese Bi-LSTM Implémentation - Code

Listing 1: Siamese Bi-LSTM

```
1 class SiameseLSTM(nn.Module):
2     def __init__(self, ntokens, ninp, nhid, nlayers):
3         super(SiameseLSTM, self).__init__()
4         self.encoder = nn.Embedding(ntokens, ninp)
5         self.rnn      = nn.LSTM(ninp, nhid, nlayers,
6                                 batch_first=True,
7                                 bidirectional=True) # Bi-LSTM
8
9     def encode(self, x, lengths):
10        emb = self.encoder(x) # (batch, seq, ninp)
11        packed = pack_padded_sequence(
12            emb,
13            lengths.cpu(),
14            batch_first=True,
15            enforce_sorted=False
16        )
17        out, (h, c) = self.rnn(emb) # h: (2*nlayers, batch, nhid)
18        # Concatene le dernier etat avant + arriere
19        h_fwd = h[-2] # (batch, nhid)
20        h_bwd = h[-1] # (batch, nhid)
21        return torch.cat([h_fwd, h_bwd], dim=1) # (batch, nhid*2)
22
23    def forward(self, cv, cv_len, job, job_len):
24        h_cv = self.encode(cv, cv_len) # (batch, nhid*2)
25        h_job = self.encode(job, job_len) # (batch, nhid*2)
26        dist = torch.norm(h_cv - h_job, dim=1) # distance entre les deux sorties
27        return dist
```

7.2 Attention Layer

Listing 2: Attention Layer

```
1 class AttentionLayer(nn.Module):
2     def __init__(self, nembed):
3         super(AttentionLayer, self).__init__()
4         self.dk = 256
5         self.W_Q = nn.Linear(nembed, self.dk)
6         self.W_K = nn.Linear(nembed, self.dk)
7         self.W_V = nn.Linear(nembed, self.dk)
8
9     def forward(self, x): # x : (batch, seq_len, nembed)
10        Q = self.W_Q(x) # (batch, seq_len, dk)
11        K = self.W_K(x) # (batch, seq_len, dk)
12        V = self.W_V(x) # (batch, seq_len, dk)
13
14        value = torch.matmul(Q, K.transpose(-2, -1)) # (batch, seq_len, seq_len)
15        scores = value / np.sqrt(self.dk)
16        soft = F.softmax(scores, dim=-1) # (batch, seq_len, seq_len)
17
18        out = torch.matmul(soft, V) # (batch, seq_len, dk)
19        return out.mean(dim=1)
```

7.3 TF-IDF

Le TF-IDF (Term Frequency – Inverse Document Frequency) est une méthode de représentation vectorielle de texte largement utilisée en traitement automatique du langage naturel. Son objectif est d'attribuer

à chaque terme un poids qui croît avec sa fréquence dans le document considéré, mais décroît avec sa fréquence dans l'ensemble du corpus.

Term Frequency (TF) mesure la fréquence relative d'un terme dans un document donné. On suppose ici qu'un mot apparaissant souvent dans un document est important pour ce document.

$$TF(t, d) = f(t, d) \frac{f(t, d)}{\max_{t \in d} f(t, d)} \quad (2)$$

où $f(t, d)$ est le nombre d'occurrences du terme t dans le document d , divisé par la fréquence du terme le plus fréquent du document.

Cette normalisation évite de favoriser artificiellement les documents longs.

Inverse Document Frequency (IDF) pénalise les mots qui apparaissent dans un grand nombre de documents, car ils sont peu discriminants. Un mot comme « experience » ou « company » présent dans tous les documents n'apporte aucune information distinctive. À l'inverse, un terme rare comme « PyTorch » est très informatif. La formule utilisée est :

$$IDF(t) = \log \left(\frac{N + 1}{df(t) + 1} \right) + 1 \quad (3)$$

où N est le nombre total de documents et $df(t)$ le nombre de documents contenant le terme t . Le décalage de $+1$ au numérateur et au dénominateur est un lissage qui évite les divisions par zéro et atténue les effets de bord pour les termes très rares.

Le score final TF-IDF d'un terme dans un document est le produit des deux :

$$TF-IDF(t, d) = TF(t, d) \cdot IDF(t) \quad (4)$$

Chaque document est ainsi représenté comme un vecteur de dimension égale à la taille du vocabulaire, dont chaque composante correspond au score TF-IDF du terme associé. Ce vecteur est ensuite normalisé par sa norme L2 afin que la comparaison entre documents ne soit pas influencée par leur longueur.

7.4 Similarité cosinus

Une fois les documents transformés en vecteurs TF-IDF, nous calculons la similarité cosinus entre chaque CV et chaque offre d'emploi.

La similarité cosinus entre deux vecteurs A et B est définie par :

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

où :

- $A \cdot B$ est le produit scalaire des deux vecteurs ;
- $\|A\|$ et $\|B\|$ sont leurs normes euclidiennes.

Cette mesure retourne une valeur comprise entre 0 et 1 :

- proche de 1 : les documents utilisent des termes similaires ;
- proche de 0 : les documents sont très différents.

Dans notre implémentation, nous calculons une matrice complète de similarité entre tous les CV et toutes les offres :

$$S_{ij} = \text{cosine_similarity}(CV_i, Job_j)$$

Chaque élément S_{ij} représente donc le score de similarité entre un CV et une offre d'emploi.

7.5 F1-score macro

En plus de l’accuracy, nous utilisons le F1-score macro pour évaluer les performances du modèle. Cette métrique prend en compte à la fois la précision (*precision*) et le rappel (*recall*).

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Dans le cas du F1-score macro, le F1-score est calculé indépendamment pour chaque classe, puis moyenné:

$$F1_{macro} = \frac{1}{C} \sum_{i=1}^C F1_i$$

où C représente le nombre de classes.

Contrairement au F1-score pondéré, le macro F1 donne le même poids à chaque classe, ce qui permet de mieux évaluer les performances globales du modèle même en présence d’un déséquilibre entre classes positives et négatives.

References

- [1] Sachin Maheshwary and Vinod Misra. Matching resumes to jobs via deep siamese network. In *Companion Proceedings of the The Web Conference 2018 (WWW '18 Companion)*, 2018.
- [2] Liang Yao et al. Confit: Improving resume-job matching using data augmentation and contrastive learning, 2024.
- [3] J. J. Decorte et al. consultantbert: Fine-tuned siamese sentence-bert for matching jobs and job seekers, 2021.